Curs 1.
Introducere in programarea declarativa. Recursivitate

1. Programare si limbaje de programare

Limbaje

- **Procedurale** (imperative) - limbaje de nivel inalt
  - Fortran, Cobol, Algol, Pascal, C,...
  - programul este o secventa de instructiuni
  - instructiunea de atribuire si structurile de control sunt pentru controlul executiei secventiale, ramificarii si ciclarii
  - rolul programului - "ce" si "cum"
    1. sa descrie CE e de calculat
    2. sa organizeze calculul
    3. sa organizeze gestionarea memoriei

    CUM
- **Declarative** (descriptive) - limbaje de nivel foarte inalt
  - se bazeaza pe expresii
  - programele pot fi vazute ca descrieri care declara informatii despre valori, mai degraba decat instructiuni pentru determinarea valorilor sau efectelor
  - renunta la instructiuni
    1. protejeaza utilizatorii de la a face prea multe erori
    2. sunt generate din principii matematice - analiza, proiectarea, specificarea, abstractizarea
  - rolul programului - "ce" (nu "cum")
  - doua clase de limbaje declarative
    1. **limbajele functionale** (de exemplu Lisp, ML, Scheme, Haskell) se focalizeaza pe valori ale datelor descrise prin expresii cu evaluarea automata a expresiilor
    2. **limbaje logice** (de exemplu Prolog, Datalog, Parlog) care se focalizeaza pe asertiuni logice care descriu relatiile dintre valorile datelor si derivari automate de raspunsuri la intrebari
  - aplicatii in Inteligenta Artificiala - demonstrarea automata, procesarea limbajului natural si intelegerea vorbirii, sisteme expert, invatarea automata, agenti

2. Recursivitate

- mecanism general de elaborare a programelor prin care un subprogram se autoapeleaza
- recursivitatea a aparut din necesitati practice (transcrierea directa a formulelor matematice)
- exista doua tipuri de recursivitate :
  - directa
  - indirecta
- orice functie calculabila poate fi exprimata in termeni de functii recursive
- doua lucruri sunt de considerat in descrierea unui algoritm recursiv: **regula recursiva** si **conditia de iesire** din recursivitate
- avantaj al recursivitatii : text sursa extrem de scurt si foarte clar
- dezavantaj al recursivitatii : umplerea segmentului de stiva in cazul in care numarul apelurilor recursive, respectiv ai parametrilor formali si locali ai subprogramelor recursive este mare

Exemple recursivitate

- o lista este o secventa de elemente $(l_1 l_2 \dots l_n)$
- lista vida (cu 0 elemente) o notam cu $\varnothing$
- prin $\oplus$ notam operatia care adauga un element in lista

a) direct recursiv

$$creareLista(n) = \begin{cases} \varnothing & daca\ n = 0 \\ creareLista(n-1) \oplus n & altfel \end{cases}$$

b) folosind o functie auxiliara recursiva pentru crearea sublistei (i, i+1,..., n)

$$creare(i,n) = \begin{cases} \phi & daca\ i > n \\ i \oplus creare(i+1,n) & altfel \end{cases}$$

3. Pseudocod

NodLSI

e: TElement //informatia utila a nodului

urm: ↑NodLSI //adresa la care e memorat nodul urmator

LSI

prim: ↑NodLSI //adresa primului nod din lista

Funcția creeazaNodLSI(*e*)
*{**pre**: e:* TElement*}*
*{**post**: se returnează un* ↑NodLSI *conținând e ca informație utilă}*
{se alocă un spațiu de memorare pentru un NodLSI *}*
{*p*: ↑NodLSI}
aloca(*p*)
[*p*].e ← e
[*p*].urm ← NIL
{rezultatul returnat de funcție}
creeazaNodLSI ← *p*
SfFuncție

Funcția creare(*i*, *n*)
*{**post**: se returnează un* ↑NodLSI*, pointer spre capul listei înlănțuite formate }*
{ din elementele i, i+1,..., n }
Dacă *i* > *n* **atunci**
creare ← NIL
altfel
{ se alocă un spațiu de memorare pentru un NodLSI *având }*
{ informația utilă e}
q ← **creeazaNodLSI**(*i*)
{ se creează legătura între nodul q și capul listei înlănțuite }
{ formate din elementele i+1,..., n }
[*q*].urm ← **creare**(*i*+1, *n*)
creare ← *q*
SfDacă
SfFuncție

Funcția creareLista(*n*)
*{**post**: se returnează un* ↑NodLSI*, pointer spre capul listei înlănțuite formate }*
{ din elementele 1, 2,..., n }
creareLista ← **creare**(1, *n*)
SfFuncție

2. Dandu-se un numar natural n, sa se calculeze suma 1+2+3+...+n

a) direct recursiv

$$suma(n) = \begin{cases} 0 & daca\ n = 0 \\ n + suma(n-1) & altfel \end{cases}$$

b) folosind o functie auxiliara recursiva pentru calculul sumei i+(i+1)+...+n

$$suma_aux(n,i) = \begin{cases} 0 & daca\ i > n \\ i + suma(n, i+1) & altfel \end{cases}$$

3. Sa se construiasca lista obtinuta prin adaugarea unui element la sfarsitul unei liste.

$$adaug(e, l_1 l_2 \dots l_n) = \begin{cases} (e) & daca \quad l \ e \ vida \\ l_1 \oplus adaug(e, l_2 \dots l_n) & altfel \end{cases}$$

4. Sa se verifice aparitia unui element in lista

$$apare(E, l_1 l_2 \dots l_n) = \begin{cases} fals & daca \ l \ e \ vida \\ adevarat & daca \quad l_1 = E \\ apare(E, l_2 \dots l_n) & altfel \end{cases}$$

5. Sa se numere de cate ori apare un element in lista

$$nrap(E, l_1 l_2 \dots l_n) = \begin{cases} 0 & daca \ l \ e \ vida \\ 1 + nrap(E, l_2 \dots l_n) & daca \quad l_1 = E \\ nrap(E, l_2 \dots l_n) & altfel \end{cases}$$

6. Sa se verifice daca o lista numerica este multime

$$eMultime(l_1 l_2 \dots l_n) = \begin{cases} adevarat & daca \ l \ e \ vida \\ fals & daca \quad l_1 \in (l_2 .. l_n) \\ eMultime(l_2 \dots l_n) & altfel \end{cases}$$

7. Sa se construiasca lista obtinuta prin transformarea unei liste numerice in multime

$$multime(l_1 l_2 \dots l_n) = \begin{cases} \phi & daca \ l \ e \ vida \\ multime(l_2 .. l_n) & daca \quad l_1 \in (l_2 .. l_n) \\ l_1 \oplus multime(l_2 \dots l_n) & altfel \end{cases}$$

8. Sa se returneze inversa unei liste

$$invers(l_1 l_2 \dots l_n) = \begin{cases} \phi & daca \ l \ e \ vida \\ & \\ invers(l_2 .. l_n) \oplus l_1 & altfel \end{cases}$$

9. Sa se construiasca lista obtinuta prin stergerea aparitiilor unui element dintr-o lista

$$stergere(E, l_1 l_2 \dots l_n) = \begin{cases} \phi & daca \ l \ e \ vida \\ l_1 \oplus stergere(E, l_2 \dots l_n) & daca \quad l_1 \neq E \\ stergere(E, l_2 \dots l_n) & altfel \end{cases}$$

10. Sa se determine al k-lea element al unei liste (k >= 1)

$$element(l_1 l_2 \dots l_n, k) = \begin{cases} \phi & daca \ l \ e \ vida \\ l_1 & daca \ k = 1 \\ element(l_2, .. l_n, k-1) & altfel \end{cases}$$

11. Sa se determine diferenta a doua multimi reprezentate sub forma de lista

$$diferenta(l_1 l_2 \dots l_n, p_1 p_2 \dots p_m) = \begin{cases} \phi & daca \ l \ e \ vida \\ diferenta(l_2 .. l_n, p_1 p_2 \dots p_m) & daca \quad l_1 \in (p_1 p_2 \dots p_m) \\ l_1 \oplus diferenta(l_2 .. l_n, p_1 p_2 \dots p_m) & altfel \end{cases}$$